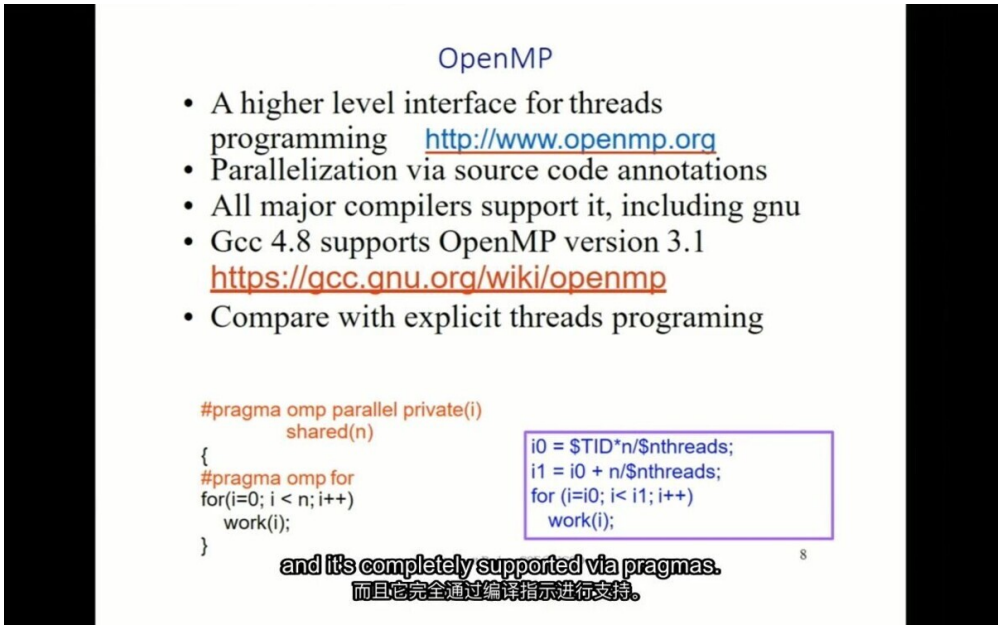
课程笔记

OpenMP：编译制导并行及其陷阱

CMU 15-418/618 并行计算机架构与编程 · Lecture 15（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 14 分 28 秒
视频链接：`https://www.bilibili.com/video/BV1Qp4y1J72A/`

目录

1 OpenMP：用编译制导表达并行 **2**
1.1 一行 pragma 就能并行化循环 . . . 2
1.2 fork-join 模型 . . . 2
1.3 本章小结 . . . 4

2 循环并行化时编译器替你做了什么 **4**
2.1 四件事 . . . 4
2.2 两种等价的写法 . . . 4
2.3 本章小结 . . . 5

3 变量作用域：默认规则是个陷阱 **5**
3.1 两条默认规则 . . . 5
3.2 为什么默认是共享反而危险 . . . 5
3.3 讲者的做法：100% 显式标注 . . . 6
3.4 本章小结 . . . 7

4 CPU 与 GPU 的调度模型之别 **7**
4.1 调度在哪里发生 . . . 7
4.2 亲和性 . . . 8
4.3 本章小结 . . . 9

5 两类让结果出错的写法 **9**
5.1 竞争与伪共享同时出现 . . . 9
5.2 跨迭代依赖 . . . 10
5.3 本章小结 . . . 10

6 循环间同步：nowait 与 reduction **10**
6.1 屏障默认与 nowait 子句 . . . 10
6.2 critical 与 reduction . . . 11
6.3 本章小结 . . . 12

7 当 OpenMP 无法并行时：重组算法 **13**
7.1 一个关键判断 . . . 13
7.2 拆成两个循环 . . . 13
7.3 一个反直觉的结论 . . . 14
7.4 矩阵向量乘法 . . . 14
7.5 本章小结 . . . 15

8 总结与延伸 **15**

1 OpenMP：用编译制导表达并行

讲者开场就说这是他最喜欢的东西之一，理由很实际：OpenMP 内建在所有 GCC 里，完全通过 pragma 支持，用起来非常自然。

1.1 一行 pragma 就能并行化循环

示例里写着 `#pragma omp parallel for`，讲者说这基本上就是并行化一个 for 循环所需的全部。

当然，他立刻补了一句关键的限定：这个循环是否应该被并行化，是另一个问题。我们要表达的是” 允许这样做”，这是给编译器的注解。

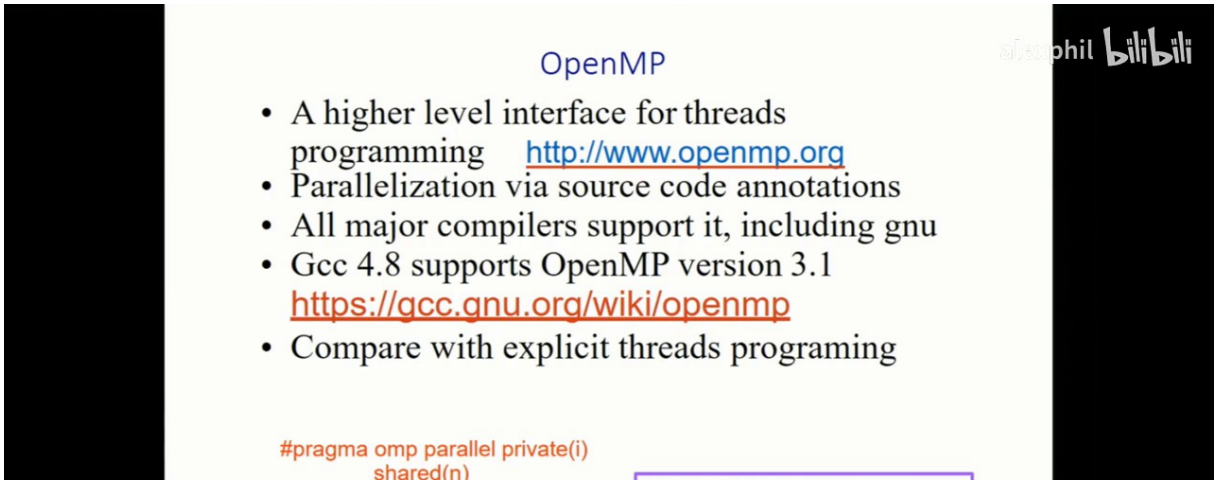


图 1: OpenMP：面向线程的高层接口[1]

pragma 是许可而非验证

`#pragma omp for` 的含义是” 我确认这些迭代可以并行”，编译器不会（也不总能）验证这个断言。因此它与第四节讨论的 forEach 属于同一类：声明控制并行，不做依赖分析。加 pragma 前必须先自己确认迭代独立。

1.2 fork-join 模型

OpenMP 采用 fork-join 模型：启动一批工作线程，它们各自独立干活，到达同步点（屏障）后重新汇合，然后再启动、再汇合。

并行区内的并行度由线程数决定：

$$P_{\text{region}} = N_{\text{threads}}$$

- N_{threads}：当前并行区中的线程数，通常等于可用核数；
- P_{region}：该区段可达到的并行度；
- 串行区段的并行度为 1，因此整体加速受 Amdahl 定律约束。

[1] 视频画面时间区间：00:01:00–00:01:15。若为裁剪图，时间区间仍对应原视频画面。

讲者强调的那条贯穿全程的蓝色线程，就是并行度为 1 时仍在运行的主线程。

朴素地理解，这意味着反复创建和回收大量线程。但讲者说编译器在这方面做得相当好，会尽量减少线程创建。

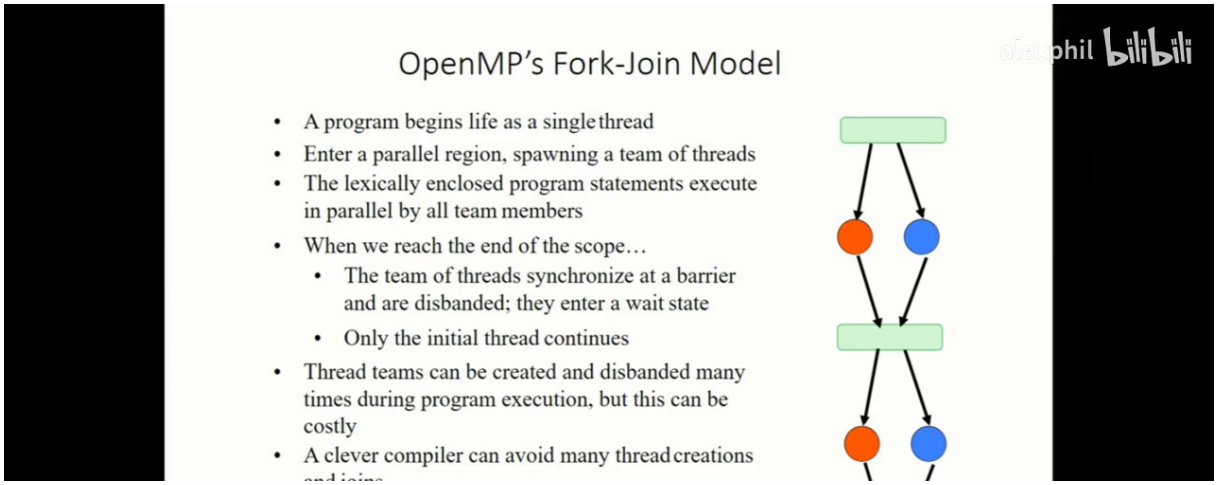


图 2: OpenMP 的 fork-join 模型：程序以单线程开始，进入并行区后派生线程[2]

主线程的存在影响屏障语义

主线程在并行区内也承担一份工作，因此屏障要求它与其他线程一样到达。这意味着若主线程在并行区内被安排去处理某段只有它能做的代码，其余线程会在屏障处等它。理解主线程贯穿始终，才能理解 OpenMP 中屏障与任务划分的边界。

讲者展示了一张并行度示意图，其中并行区之间有串行执行的区段，并特意指出有一条蓝色的线程贯穿到底。他让学生猜它是什么——答案是主线程：程序启动时的那个线程从未消失，整个过程都在。

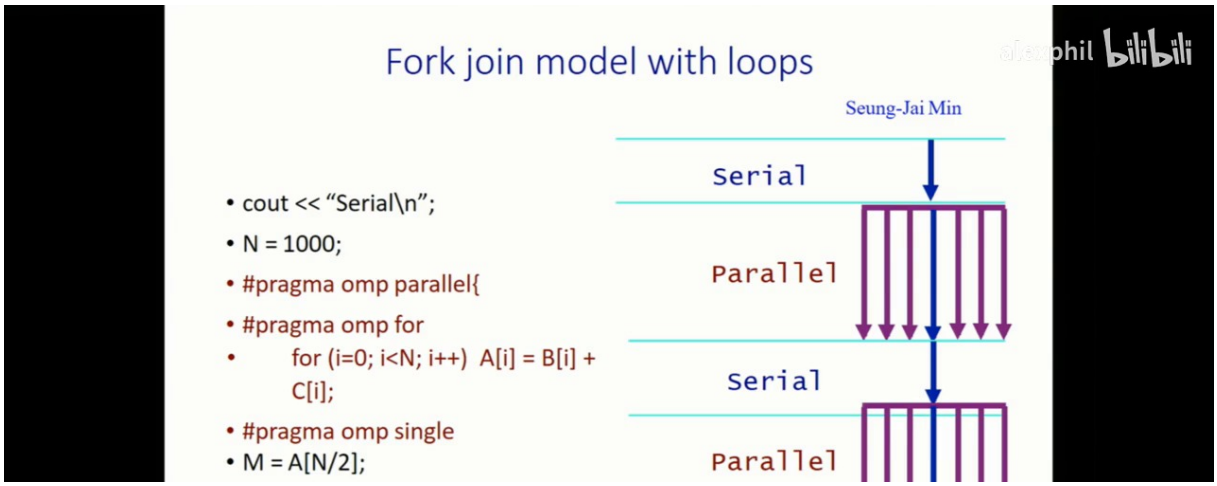


图 3: 循环上的 fork-join：并行区之间的串行段[3]

[2]视频画面时间区间：00:02:00–00:02:15。若为裁剪图，时间区间仍对应原视频画面。

[3]视频画面时间区间：00:02:30–00:02:45。若为裁剪图，时间区间仍对应原视频画面。

1.3 本章小结

OpenMP 通过 pragma 在高层表达并行，采用 fork-join 模型，主线程贯穿始终，编译器负责优化线程创建。一行 pragma 足以并行化循环，但它只是许可，不是正确性验证。

2 循环并行化时编译器替你做了什么

2.1 四件事

讲者列出翻译器自动完成的工作：生成各线程的局部循环边界，也就是替你划分；插入所需的屏障；处理循环次数不能被线程数整除时的余数；划分方式可以是静态或动态。

剩下的必须由程序员指定：哪些变量私有、哪些共享。

划分时每个线程分到的迭代数并不总能整除：

$$\text{chunk} = \left\lceil \frac{N}{T} \right\rceil, \qquad \text{last} = N - (T-1) \cdot \text{chunk}$$

- N：循环迭代总数；
- T：线程数；
- chunk：每个线程分到的迭代数，向上取整；
- last：最后一个线程实际处理的迭代数，可能少于 chunk。

编译器会自动生成这些边界，但最后一个线程的负载因此偏轻。

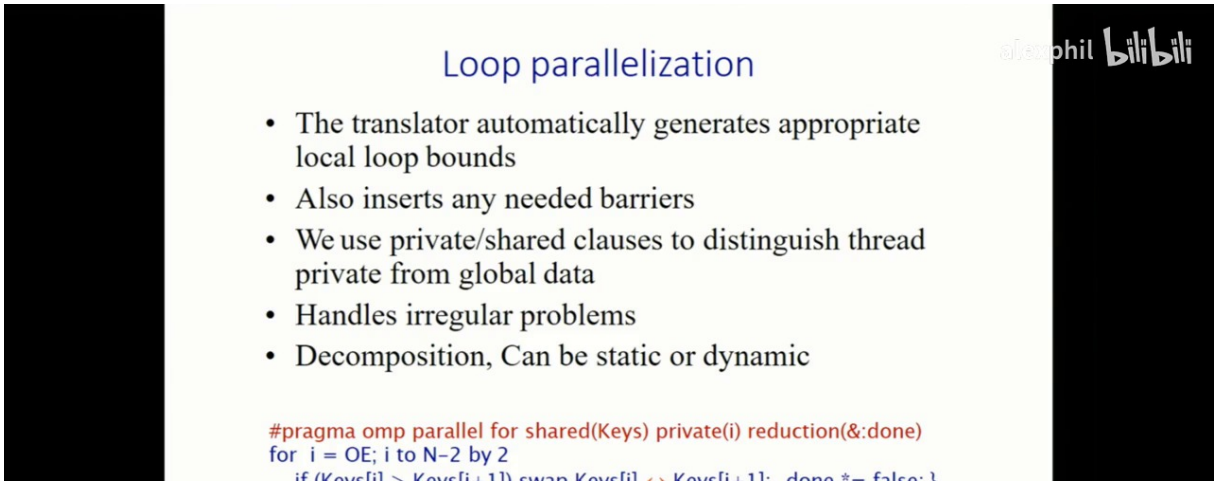


图 4: 循环并行化：翻译器自动生成局部边界、插入屏障、处理余数[4]

2.2 两种等价的写法

讲者指出 `omp for` 与 `omp parallel for` 表达的是同一件事，编译器与翻译器得到的信息相同。

[4] 视频画面时间区间：00:04:30–00:04:45。若为裁剪图，时间区间仍对应原视频画面。

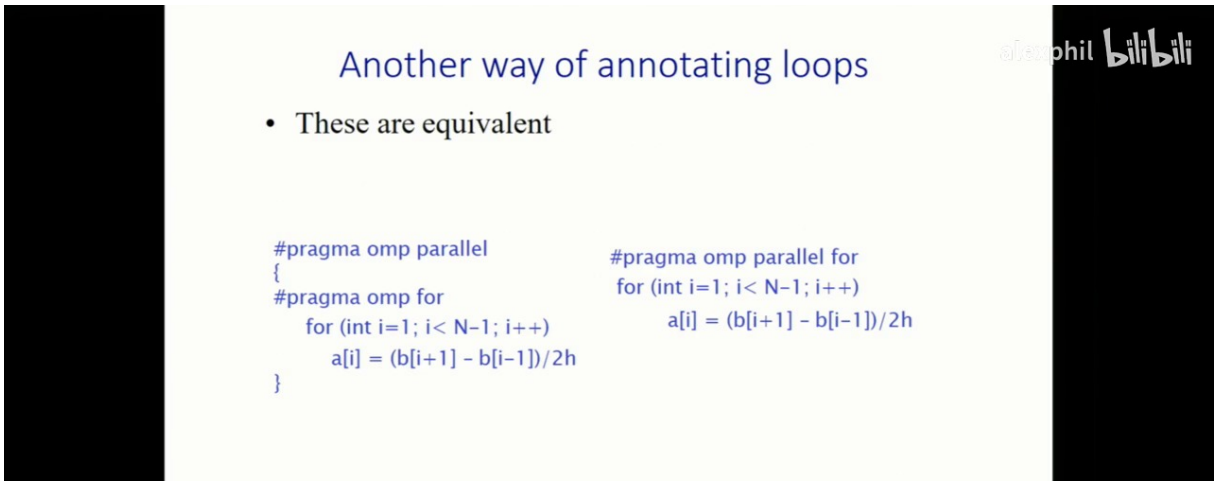


图 5: 两种等价的循环标注写法[5]

合并写法的隐含边界

`omp parallel for` 等于”开并行区”加”在区内划分循环”两步合一，因此它自带一个隐式的并行区边界。若在一个已开的并行区内部再写 `omp parallel for`，会嵌套出新的并行区，通常不是本意。这正是两种写法虽等价却不可随意互换的原因。

2.3 本章小结

编译器负责划分、插屏障、处理余数，并把私有与共享的判断留给程序员。两种 pragma 写法等价但边界含义不同。

那个被留下的判断正是最容易出错的地方，下一章专门讨论。

3 变量作用域：默认规则是个陷阱

3.1 两条默认规则

讲者给出 OpenMP 的默认约定：在并行区之前声明的变量，默认被所有线程共享；在并行区内部声明的变量，默认私有。

3.2 为什么默认是共享反而危险

讲者请学生想一件事：一旦开启并行，程序就变成多份实例。此时在”共享”与”私有”之间存在一个显然的选择。

他的判断很明确：私有才应该是常态，因为共享意味着并发控制问题——必须以某种方式管理，而这实际上会降低并行度。

[5]视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

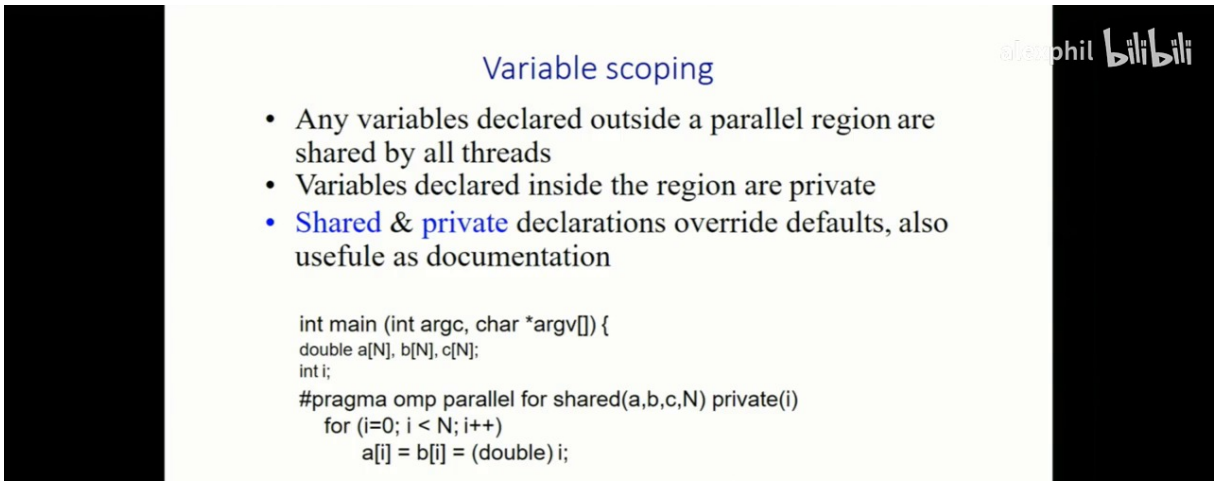


图 6: 变量作用域：并行区外声明的变量默认共享，区内声明的默认私有[6]

也就是说，默认规则把最危险的选择设成了默认值。

3.3 讲者的做法：100% 显式标注

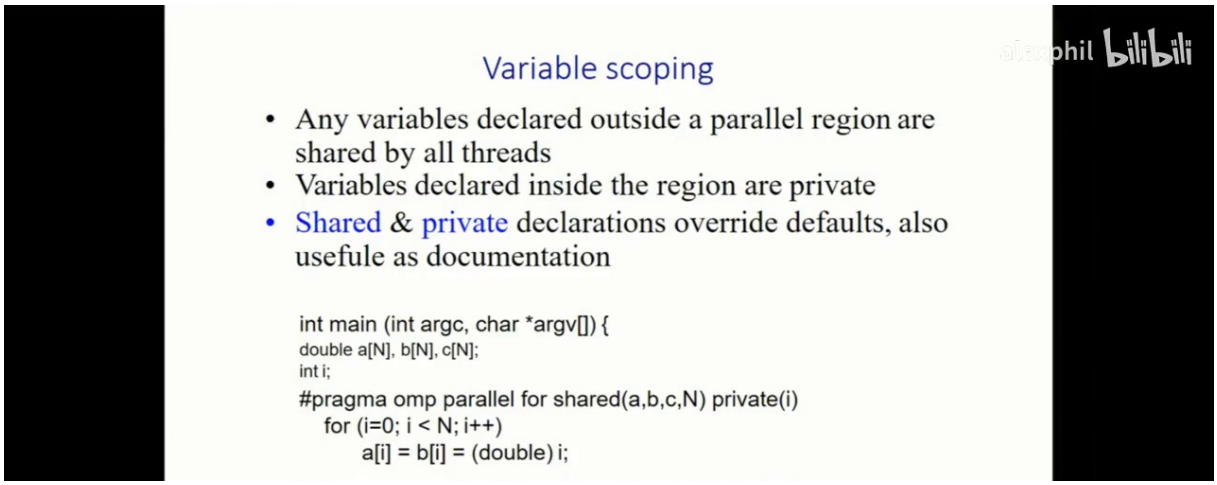


图 7: 变量作用域续：shared 与 private 子句的用法[7]

讲者说他写 OpenMP 代码时，即使不需要也会显式写 private，即使不需要也会显式写 shared。他给出的比例是 100% 的时间都这样做。

[6] 视频画面时间区间：00:06:30–00:06:45。若为裁剪图，时间区间仍对应原视频画面。

[7] 视频画面时间区间：00:09:30–00:09:45。若为裁剪图，时间区间仍对应原视频画面。

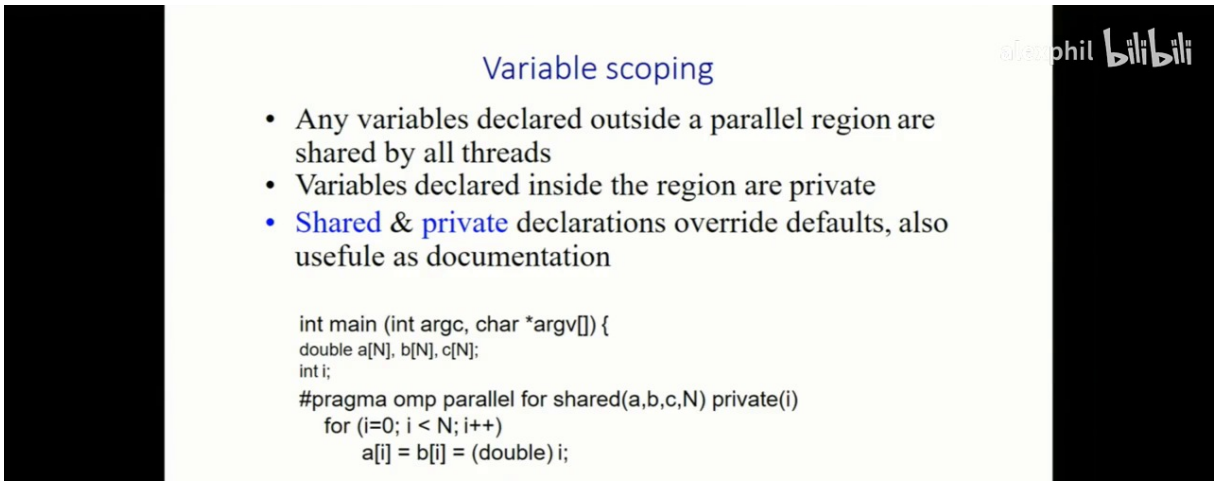


图 8: 变量作用域续：为何必须显式标注[8]

理由是两条。第一，若有人做维护性改动，把某个声明挪了位置——他以为自己在清理代码——却不知道这对 OpenMP 的含义，就会出问题。第二，看到一段并行代码时，我想直接知道我的假设是什么。要么去追溯变量在哪里声明，要么让它直接标出来。

显式标注把隐式契约变成文本契约

默认规则要求读者追溯变量声明位置才能推断共享性，而这对跨文件、宏展开或后续维护都不可靠。显式写出 shared 与 private，等于把并行区的假设写进代码本身。代价是冗长，收益是任何改动都会被阅读者立刻看到。

他问哪一种是正确答案，答案是后者。

默认规则让”搬动一行声明”变成语义变更

在普通 C 代码里把变量声明从函数开头移到循环内部，通常只是风格问题。但在 OpenMP 并行区里，这会把该变量从共享变成私有（或反之），可能让程序的结果完全改变，而且不会有任何编译警告。这正是讲者主张 100% 显式标注的根本原因。

3.4 本章小结

OpenMP 的默认规则是” 区外声明共享、区内声明私有”，而共享恰恰是需要并发控制的危险选项。因此讲者主张始终显式标注：既防止维护者无意改动引入语义变化，也让并行区的假设对读者直接可见。

4 CPU 与 GPU 的调度模型之别

讲者在回答学生提问时澄清了一件容易被混淆的事。

4.1 调度在哪里发生

在 GPU 上，warp 的调度由硬件负责：硬件持有这些 warp，逐条以锁步方式执行其中一个。

[8]视频画面时间区间：00:11:30–00:11:45。若为裁剪图，时间区间仍对应原视频画面。

回到通用 CPU 模型后，调度不再以那种方式在芯片上发生，而是由操作系统负责。OS 持有一张待运行线程的列表，有些线程是进程的唯一线程，有些是更大任务的一部分。定时器设备周期性中断，操作系统调度器运行，决定此刻在发生什么、想让什么发生、想让哪个让出。



图 9: CPU 与 GPU 的调度模型不同：前者由操作系统调度，后者由硬件调度[9]

4.2 亲和性

讲者提到一个相关概念：亲和性，即让某组线程绑定在同一个核上运行，从而利用它们之间的缓存关系。

用 POSIX 线程库时可以指定线程如何成组，以利用缓存层次。理由是同一进程内的多个线程共享同一内存上下文。若共享同一内存上下文，它们的缓存复用就有价值。



图 10: 亲和性：让相关线程共享缓存层次[10]

[9] 视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

[10] 视频画面时间区间：00:14:30–00:14:45。若为裁剪图，时间区间仍对应原视频画面。

亲和性是把”映射”这一步交给程序员

第 4 讲把线程映射到硬件单元列为四个步骤中的最后一步，通常交给系统。亲和性就是把这一步的决策权收回自己手里。当线程之间共享数据、需要共享缓存时，把它们绑到同一个插槽或用近邻核意味着更低延迟；反之若各线程独立工作，分散到不同插槽能获得更大总缓存。

4.3 本章小结

CPU 的线程调度由操作系统通过时钟中断完成，GPU 的 warp 调度由硬件完成。亲和性让程序员可以干预”线程落在哪个核上”，以利用共享缓存或避免跨插槽通信。

这些是背景知识。回到 OpenMP 本身，真正的陷阱在于依赖。

5 两类让结果出错的写法

5.1 竞争与伪共享同时出现

讲者展示了一个循环，其中各线程写相邻的元素。学生一眼指出两个问题。

第一是数据竞争：并行 for 的执行顺序没有任何保证，因此某个线程读到前一项时，完全不知道负责它的线程是否已经执行过。

第二是伪共享：这些元素是相邻的字，一个线程写就会作废另一个的缓存行，所以所有核都必须反复把这些缓存行拉回来。

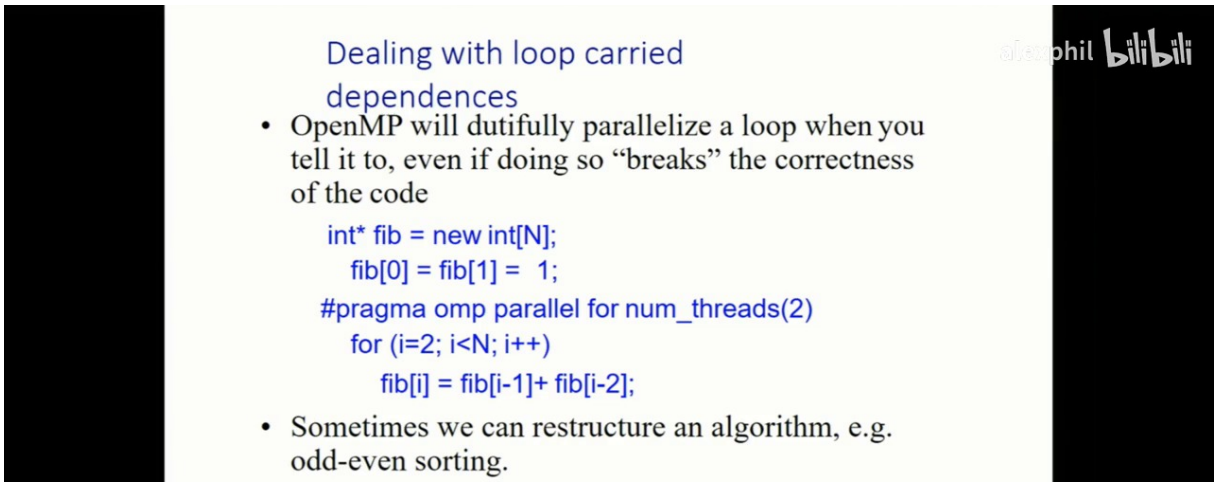


图 11: 相邻元素的并发写：竞争与伪共享同时发生[11]

同一段代码可能同时犯两类错误

数据竞争影响正确性，伪共享只影响性能，但它们可能源于同一处写法：多个线程处理相邻数据。此时既要修正确性（拆分依赖），也要修布局（避免同行争用）。只修一侧，程序要么还是错的，要么仍然很慢。

[11] 视频画面时间区间：00:21:30–00:21:45。若为裁剪图，时间区间仍对应原视频画面。

5.2 跨迭代依赖

讲者把问题抽象为跨迭代依赖。这类循环回读前一项、前两项，因此唯一能成功执行的调度就是串行。

而 OpenMP 会照你说的去做：你要求并行化，它就并行化，然后结果就错了。

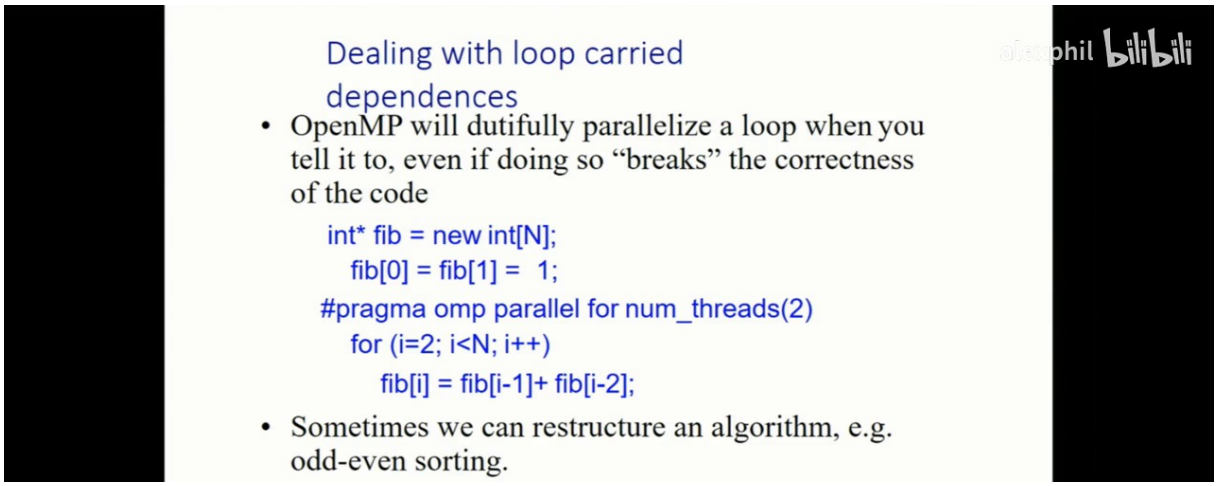


图 12: 跨迭代依赖：OpenMP 会照做并行化，但结果是错的[12]

讲者替 OpenMP 说了句公道话：它只是照你要的做，而且做得完全正确——错的是我们的要求。他还说明编译器有时能发现简单依赖并给出警告，但它并不真的知道全部情况，因此正确性责任在程序员。

不要指望编译器替你发现依赖

依赖分析在存在指针别名、间接寻址或不规则索引时会失效。OpenMP 有时给出警告，有时静默通过。因此" 编译没报错" 完全不构成" 依赖已被处理" 的证据。

5.3 本章小结

跨迭代依赖使唯一正确调度是串行，而 OpenMP 会照做，因此结果错误而责任在程序员。同一段代码还可能同时犯两类错误：相邻元素并发写既引发竞争（正确性）又引发伪共享（性能）。编译器有时能给出警告，但依赖分析在指针别名与间接索引下会失效。

除了循环本身，循环之间的同步也是一个独立的风险来源。

6 循环间同步：nowait 与 reduction

6.1 屏障默认与 nowait 子句

讲者接着解释一个默认行为：并行 for 结束时，OpenMP 会插入屏障，也就是这个循环必须全部完成后才能往下走。

若不想在循环末尾同步，可以加 nowait 子句，让线程完成即离开、直接执行后面的代码。

[12] 视频画面时间区间：00:21:40–00:21:55。若为裁剪图，时间区间仍对应原视频画面。

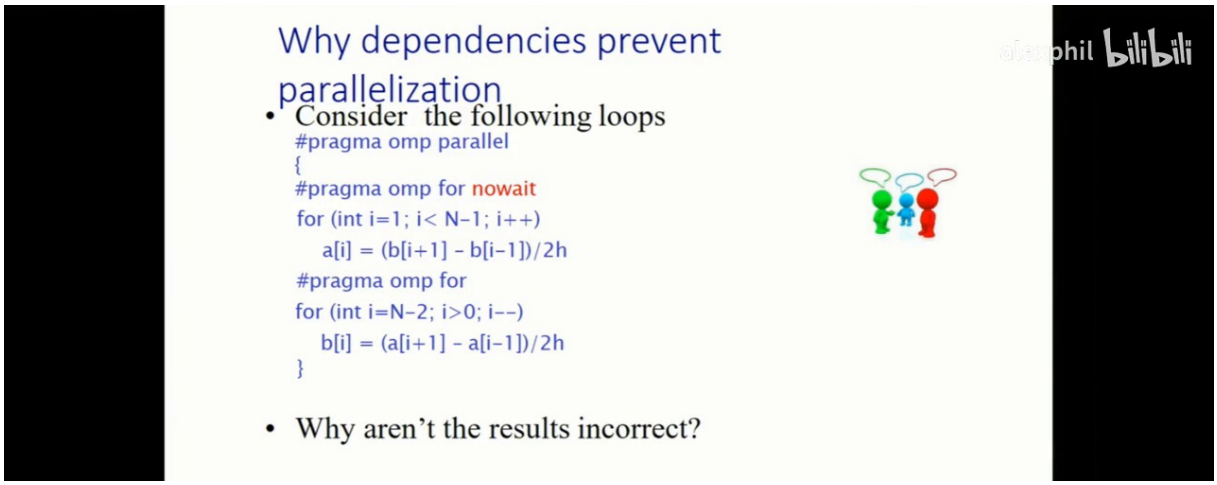


图 13: nowait 子句：去掉循环末尾的屏障[13]

他给出一个被”故意下毒”的例子：第一个循环带 nowait，第二个循环没有。第一个循环基于读 B 的结果给 A 赋值，第二个循环基于读 A 的结果给 B 赋值。两个循环之间存在依赖，而 nowait 恰好去掉了它们之间的同步。

nowait 与循环间依赖组合时最危险

两个相邻循环若存在数据流（前一循环写、后一循环读），默认屏障正是保证它们的正确性。加 nowait 后，线程可能在别的线程完成第一循环前就开始第二循环，读到旧值。这类错误的特征是偶尔出错、规模小时往往看不出来。

讲者的提示很明确：这个程序的唯一正确调度就是串行。

6.2 critical 与 reduction

讲者还讨论了 critical 区：需要保护共享更新时，把相关语句放进临界区。以及 reduction 子句：在某些应用里需要把一批值归约成单个全局值，OpenMP 提供声明式的支持。

其语义等价于部分和加最终合并：

$$\text{sum} = \bigoplus_{t=1}^{T} \text{priv}_t$$

- priv_t：第 t 个线程的私有累加值；
- $\bigoplus$：由 reduction 子句指定的结合运算（如加法、最大值）；
- T：线程数。

因为各线程只更新自己的私有副本，循环体内不需要任何加锁。

[13] 视频画面时间区间：00:26:00–00:26:15。若为裁剪图，时间区间仍对应原视频画面。

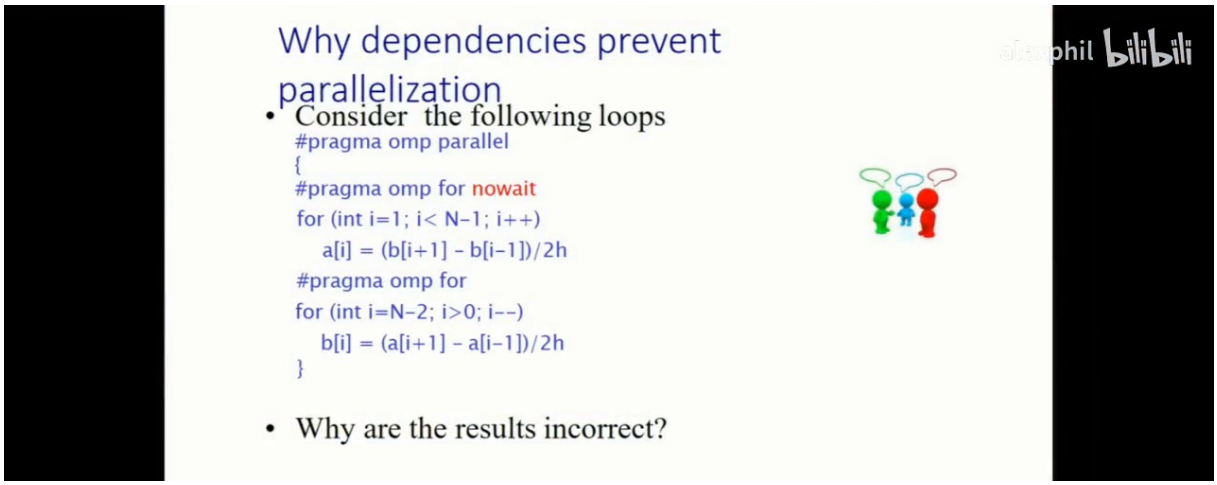


图 14: critical 区：保护共享更新[14]

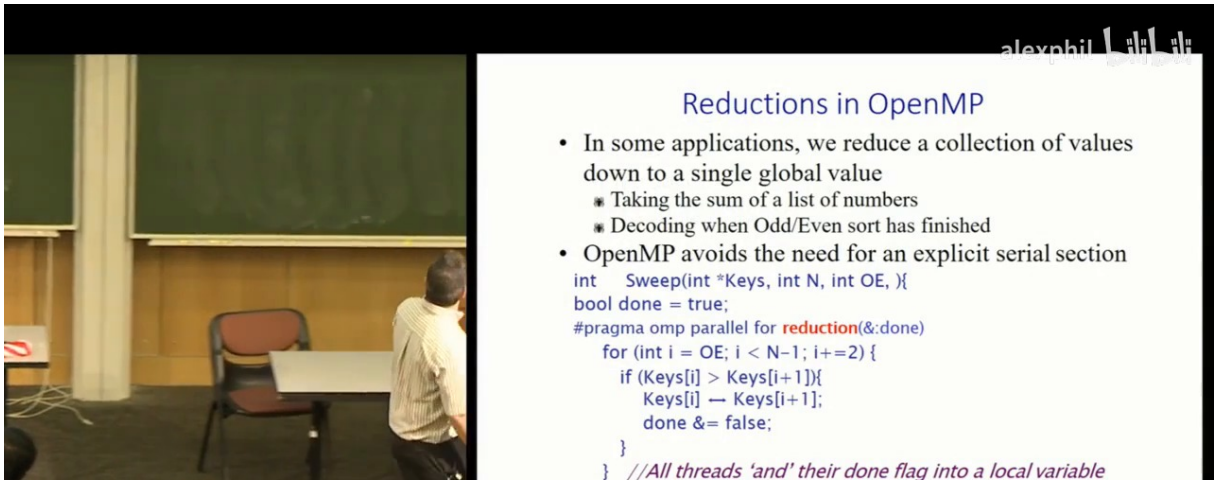


图 15: reduction 子句：自动汇总各迭代结果[15]

reduction 等价于部分和加最终合并

`reduction(+:sum)` 的语义是每个线程维护一份私有副本各自累加，循环结束后再合并。这与第 4 讲" 部分和加串行合并" 是同一件事，只是由编译器生成。因此它既避免了竞争，也避免了每次更新都加锁的开销。

6.3 本章小结

两类典型错误：相邻元素的并发写同时引发竞争与伪共享；跨迭代依赖使唯一正确调度是串行，而 OpenMP 会照做。默认的循环末尾屏障是保护措施，nowait 去掉它时必须确认后续代码不依赖该循环的结果。critical 保护共享更新，reduction 用私有副本加合并替代它。

这些陷阱都指向同一个结论：能并行化什么，取决于代码的真实依赖结构。

[14] 视频画面时间区间：00:31:00–00:31:15。若为裁剪图，时间区间仍对应原视频画面。

[15] 视频画面时间区间：00:55:00–00:55:15。若为裁剪图，时间区间仍对应原视频画面。

7 当 OpenMP 无法并行时：重组算法

7.1 一个关键判断

讲者在讨论重组时给出本讲最有价值的一句话：OpenMP 无法并行化某段代码，并不意味着人也无法并行化它。

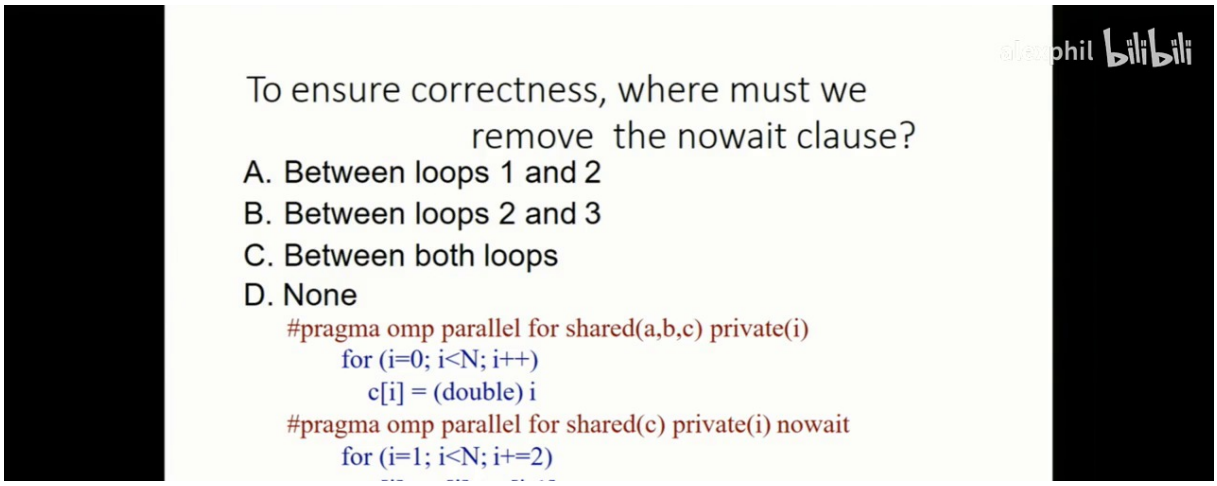


图 16: 重组算法：拆成两个循环解开依赖[16]

只要理解代码在做什么，就可以做更高层的重组，得到相同的结果但换成可并行的计算方式。

7.2 拆成两个循环

举例：一个循环里，`B[i]` 依赖于 `C[i]` 的旧值，而 `C[i]` 又依赖前一次迭代的结果，于是既有”读到未更新值”的风险，也有”用未来值参与当前计算”的风险——两个方向的问题同时存在。

解法是把它拆成两个循环，从而解开纠缠。

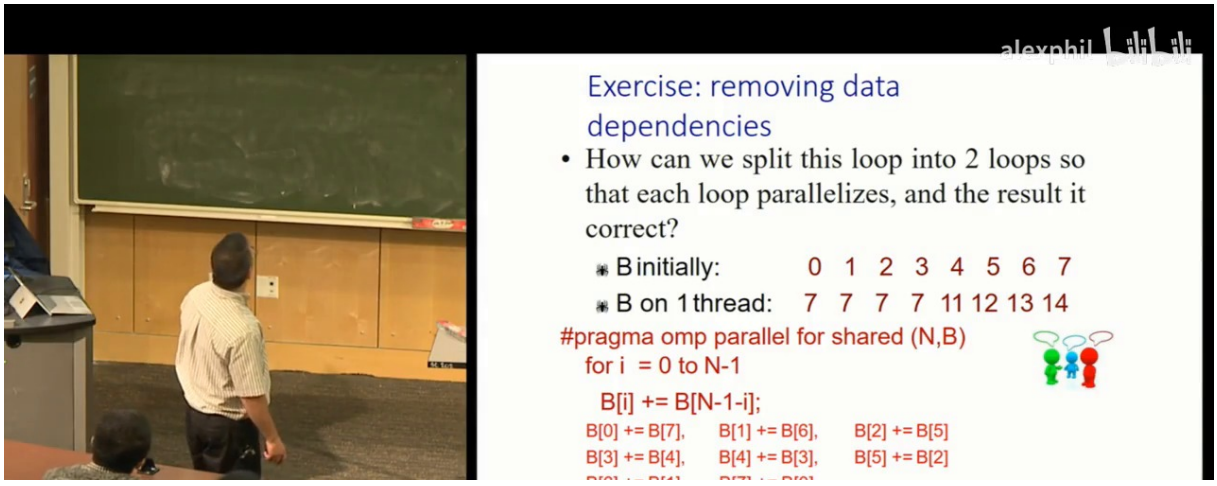


图 17: 从两端向中间配对遍历：等价于单向遍历但可并行[17]

[16] 视频画面时间区间：00:44:50–00:45:05。若为裁剪图，时间区间仍对应原视频画面。

[17] 视频画面时间区间：00:50:00–00:50:15。若为裁剪图，时间区间仍对应原视频画面。

算法重组与并行化无关，它改变的是计算的组织方式

拆成两个循环后，输出数组的内容完全不变，但依赖关系从”链式”变成”两个独立阶段”。这不是把串行代码伪装成并行，而是找到一个数学上等价、依赖结构更好的计算顺序。这类重组需要理解问题本身，而不是熟悉 OpenMP 语法。

7.3 一个反直觉的结论

讲者给出另一个例子：`B[i]` 加上 $n-1-i$，即从外向内成对处理——索引 0 与最后一个配成一对，索引 1 与倒数第二个配成一对，逐步向中间收拢。

把这种对称性显式写出来后，原本的循环可以被改写。讲者问学生这么做如何解决问题，并在讨论后确认了一个反直觉的结论：如果去掉之前那个 nowait，反而更并行也更正确。因为前半段可以并行，而之前那样写则完全无法并行。从并行度角度看，这相当于把整段串行改成了「前半段并行、后半段串行」的 2 个阶段结构。

nowait 不是越多越好

nowait 消除的等待在某些场合是必须的同步。为了”让线程早点走”而加 nowait，常常把可以安全并行的结构变成错误结构。判断依据是后续代码是否依赖这个循环的输出，而不是线程是否能早点结束。

7.4 矩阵向量乘法

讲者最后用矩阵向量乘法作为经典案例说明这类问题在实际应用中的形态。这个例子的价值在于它同时具备两个特征：外层对输出向量的每个元素独立，因此天然可并行；而每个输出元素需要遍历整行，因此访存模式与缓存复用直接决定实际收益。讲者提醒，正因如此，同一份矩阵向量乘法按行并行与按列并行在性能上差别很大——前者每个线程顺序扫描一行，局部性良好；后者每个线程跨行取数，每一列都需要跳读。这与第 8 讲分块形状的结论一致：并行化的关键是数据布局与访问顺序，而不只是把循环拆开。

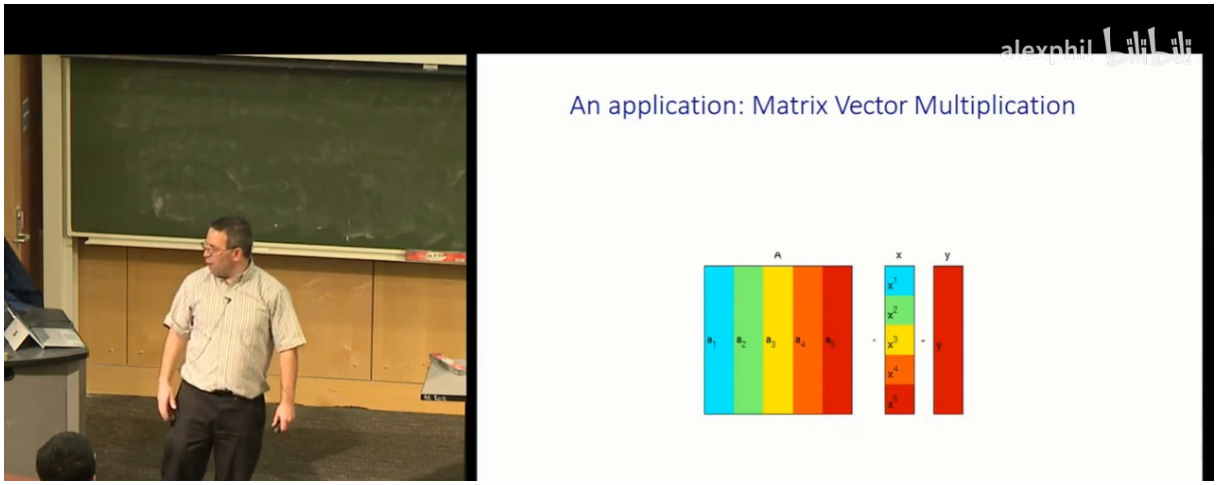


图 18: 矩阵向量乘法：一个应用案例[18]

[18] 视频画面时间区间：01:00:00–01:00:15。若为裁剪图，时间区间仍对应原视频画面。

7.5 本章小结

OpenMP 无法并行化不代表人无法并行化。通过拆分为两阶段、显式化对称配对等方式重组算法，可以在输出完全不变的前提下得到可并行的依赖结构。这类工作依赖对问题的理解，而不是语法技巧。反过来，滥用 nowait 会把安全的结构变危险。

8 总结与延伸

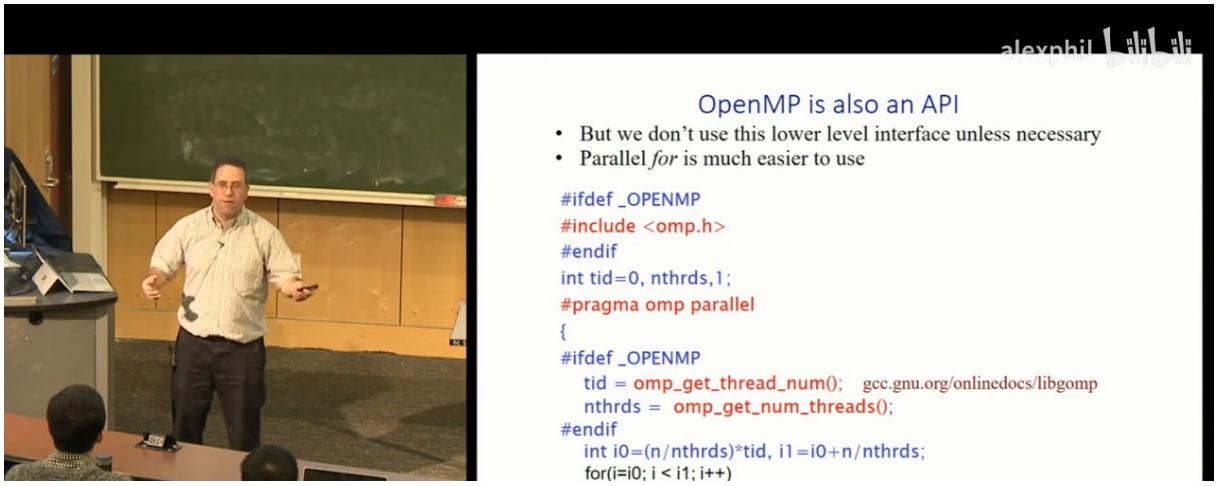


图 19: OpenMP 模型的适用边界：并行 for 最易用，更细控制需要更低层接口[19]

另外，讲者也说明 OpenMP 同时提供了一套底层的 API，但除非必要，他们并不使用这套较低层的接口，因为并行 for 更易用。这一点值得记住：同一个框架往往同时提供声明式与命令式两种入口，前者适合结构规整的循环，后者适合需要手工控制线程与同步的场景。选择哪一档，取决于代码的规整程度，而不是取决于哪种写法看起来更专业。

讲者在结尾坦率地评价了这个模型的边界：它是一个很快就会变得纠缠的模型。它确实是为”派生一堆线程、并行做一件事、同步、再并行、再同步”而设计的。如果你要做别的事情，更低层的模型可能更合适。但他也承认这个模型对很多问题非常好用且干净，他自己就很喜欢它。

把全讲串起来，OpenMP 用 pragma 在高层表达并行，采用 fork-join 模型，主线程贯穿始终。编译器自动划分循环、插入屏障、处理余数，但把私有与共享的判断留给程序员——而默认规则恰恰把最危险的选项（共享）设成了默认值，因此讲者主张 100% 显式标注。真正的陷阱有两类：相邻元素的并发写同时引发竞争与伪共享；跨迭代依赖使唯一正确调度是串行，而 OpenMP 会照做。循环末尾的屏障是保护措施，nowait 去掉它时必须确认后续不依赖结果。critical 保护共享更新，reduction 用私有副本加合并替代。而当 OpenMP 无法并行时，正确的方向是重组算法、拆成两阶段、显式化对称性，而不是硬加 pragma 或加 nowait。

本讲义在讲者材料之上做的压缩，有四点需要与讲者原话区分：

- 默认规则的危险不来自它本身错，而来自它让语义依赖于声明位置。同一个变量在函数开头和循环内部声明，行为不同且无警告。显式标注不是为了风格一致，而是为了把语义从”位置隐含”变成”文本可见”。
- pragma 与 forEach 属于同一类原语。它们声明控制并行而不验证依赖，因此正确性责任完全在程序员。这一判断在第 4 讲就已建立，这里只是换了一个语法。

[19] 视频画面时间区间：01:12:10–01:12:25。若为裁剪图，时间区间仍对应原视频画面。

- nowait 的效果是双向的。它既可能修复多余的同步，也可能破坏必需的同步。判据只有一个：后续代码是否依赖该循环的全部结果。
- 重组算法是并行工程的主要创造性工作。语法层面的 pragma 只是表达手段，而决定能达到多少并行度的是依赖结构。因此优化瓶颈往往不在” 怎么写 pragma”，而在” 怎么重排计算”。

这堂课的边界在于它是工具导论：讲者多次表示 OpenMP 的细节很多，学生需要在作业中体会。因此本文给出的是语义规则与陷阱清单，不含完整的子句参考。

两个可以直接从本讲数据提出的下一步问题：其一，既然默认规则危险，那么把 OpenMP 代码库整体改成显式标注，是否会掩盖本来应当被发现的依赖错误？其二，重组算法得到的是数学等价的另一个计算顺序，那么在浮点运算下” 等价” 是否严格成立，误差累积是否可能改变结果？